



Qwen2.5-Coder-7B-SFT:

——学术论文到 PyTorch 代码的 LoRA 微调

——微调模型子项目报告

成员：但杰、蒋佳媛、高鑫彤、徐媛

讲师：张瑶

2026.6.3



目 录

- 1 研究背景与架构定位
- 2 数据工程与合规验证
- 3 核心微调与训练结果
- 4 增量工作和交付结果
- 5 未来展望与局限分析



1

研究背景与架构定位

1.1 研究背景与核心痛点

研究背景：

在论文复现工作中，基于论文生成可执行的算法代码是核心问题。



语义鸿沟缺失

学术长文本与多公式

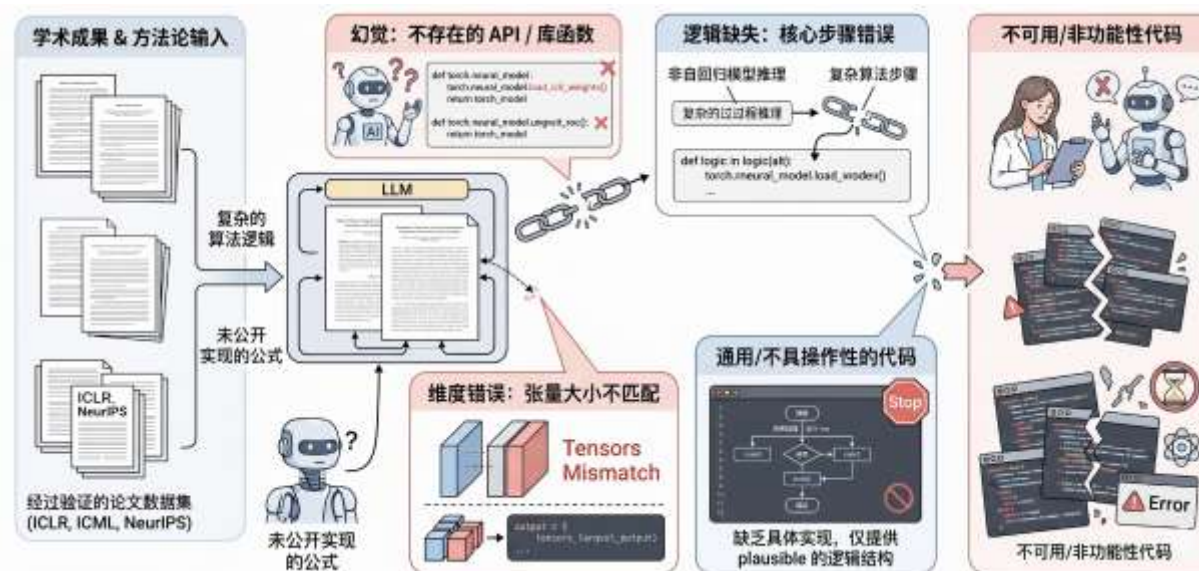
→ 无法生成结构化代码



缺乏结构遵循

松散代码片段

→ 无法生成一致的代码



通用大模型

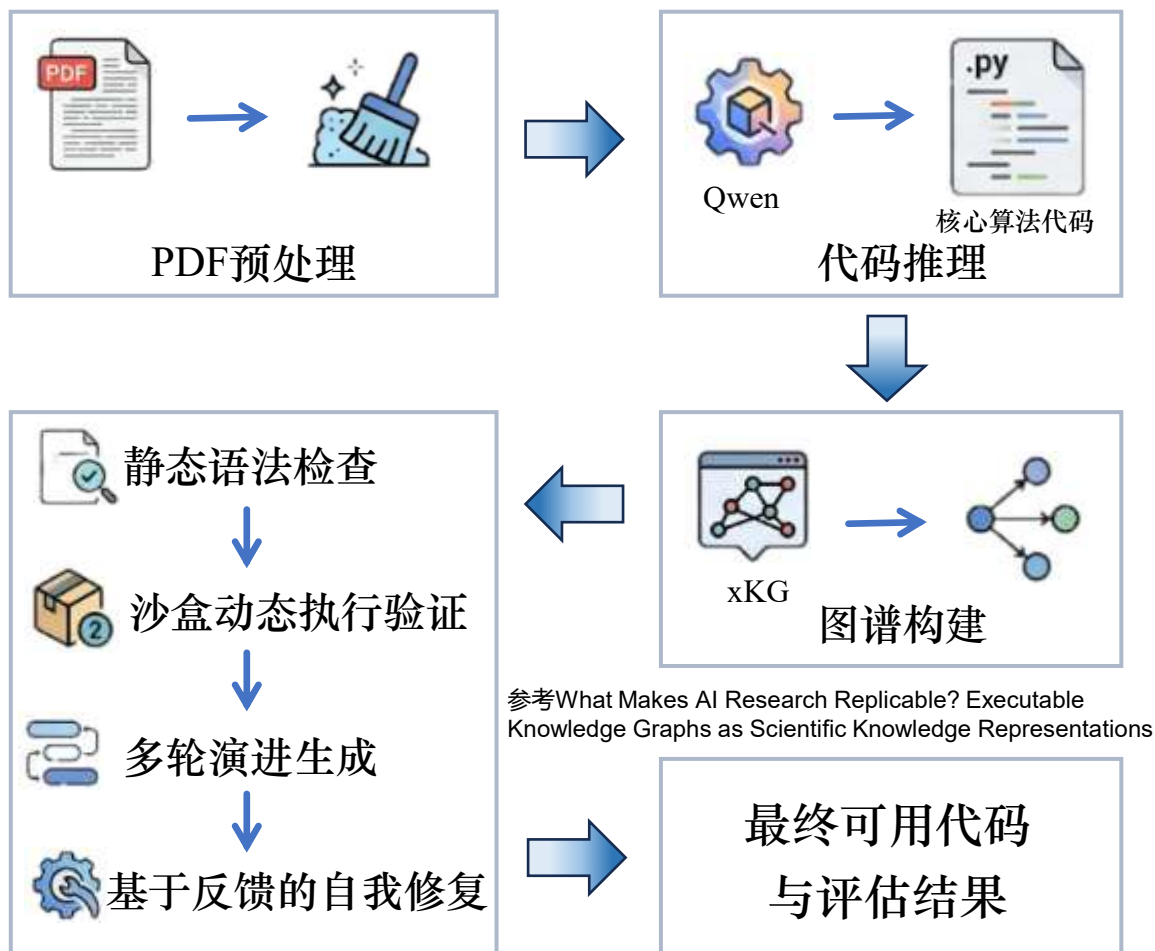
- ❌ 混乱、报错频发
- ❌ 单一散乱代码片段
- ❌ 需大量人工介入修补

本项目目标

- ✅ 精准、高度结构化
- ✅ 完整的多文件工程级输出
- ✅ 自动化直接可执行

1.2 Paper2XAgent全局系统架构

➤ Paper2XAgent全端到端智能体系统架构



➤ 技术使命与模型选取依据



核心使命：

实现高忠实度和高可运行性的端到端代码初版生成



技术选型考量：

核心选型：Qwen2.5-Coder-7B模型

- 多语言与长文本
- 基座工程能力
- 计算资源可控



效能预估：

预期效能：高效端到端推理过程

10min（极速微调） **vLLM+**（高效并发推理能力）

论文 → 核心代码 → 沙盒验证 → 自我修复 → 生成代码



2

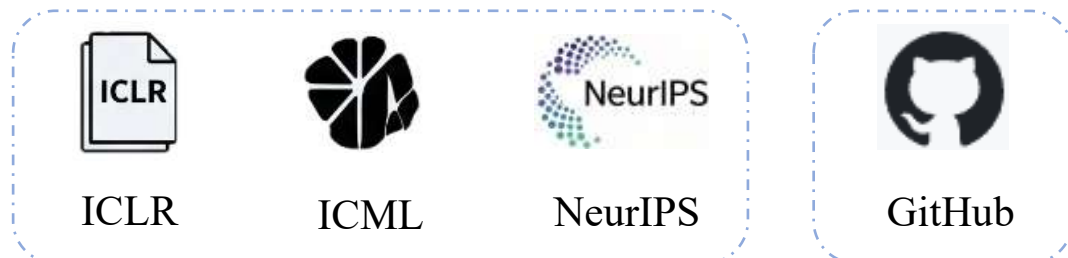
数据工程与合规验证



2.1 数据源获取与数据构建

➤ 数据来源与规模

■ 来源:



- 2024、2025年顶级机器学习会议论文
- 对应的GitHub开源仓库

■ 数据集规模:

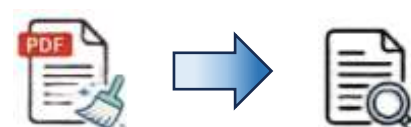
类型	数量	论文年份
训练集	90条	2024
测试集	75条	2025

注: 测试集来自2025年论文, 与训练集严格无交集。

参考文献: 《Paper2Code: Automating Code Generation from Scientific Papers in Machine Learning》、《Survey of Automatic Text Summarization Based on Deep Learning》

➤ 训练数据构建流程

1 第一阶段: 文本清洗



- 基于LLM的摘要总结策略
- 匹配原文内容补全细节

高重要性 method、model 等

中等重要性 experiment、result等

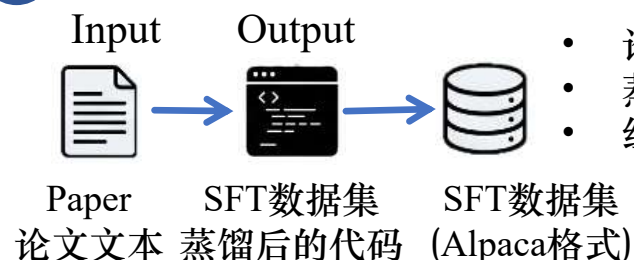
低重要性 introduction、conclusion 等

2 第二阶段: 代码提取



- 基于origen策略构建agent流抽取核心代码 (详见4.1节)
- 克隆GitHub仓库
- 提取所有Python文件, 进行蒸馏

3 第三阶段: 格式组装



- 论文文本作为input
- 蒸馏后的代码作为output
- 组装成Alpaca格式



2.2 许可合规以及训练样本结构

➤ 数据合规体系

许可情况与合规体系



论文数据：公开出版学术会议论文



代码数据：GitHub开源仓库，使用MIT 或 Apache-2.0 许可证



基座模型：Qwen2.5-Coder-7B-Instruct
使用 Tongyi Qianwen License



训练框架：LLaMA-Factory 使用
Apache-2.0 许可证



时序隔离：训练集来自2024年论文，
测试集2025，严格分离

➤ Alpaca训练样本结构

1. Instruction（指令）

角色设定：“你是一个将学术论文转换为
PyTorch 代码的助手”

注：仅作参考，实际提示词更加完整

2. Input（输入）

论文文本的核心章节

3. Output（输出）

标准代码：符合相应格式的蒸馏代码

模型的学习目标



2.3 数据有效性验证

➤ 构建数据集对应的测试指标

平均XPR（可执行率，取值 $[0,1]$ ）

平均SYR（语法正确率，取值 $[0,1]$ ）

	平均XPR	平均SYR
训练集	0.6556	0.8889
评测集	0.6472	1.0

➤ 流程指示：核心文本与核心代码对齐与检验

Step 1: 基于Agent的自动校验



- ✓ 根据确认核心内容的方法及指标构建Agent分层注入prompt校验
- ✓ 全量数据集，自动测试验证

Step 2: 人工抽样检查



- ✓ 分层随机抽样、进行人工校验
- ✓ 不同年代和不同会议论文抽样



3

核心微调与训练方案



3.1 消融实验：基座模型对比（固定 4-bit QLoRA）

在相同4-bit QLoRA微调约束下，评估不同预训练架构的学术代码生成能力

➤ 控制变量：

训练数据：相同的 10 条 Alpaca 格式数据

训练配置：num_train_epochs=8, lr=2e-4,
cutoff_len=2048, lora_target=all

评估集：相同的 10 篇论文
(ICLR/ICML/NeurIPS)

评估指标：XPR, SYR, CodeBERTScore, Overall
Score

➤ 实验结果：

基座模型	SYR	XPR	CodeBERTScore	Overall	峰值显存
Qwen2.5-Coder-7B-Instruct	34.7%	14.7%	0.444	20.5%	13.2GB
Qwen2.5-7B-Instruct	30.6%	15.2%	0.467	19.8%	13.5GB
DeepSeek-Coder-V2-Lite (16B MoE)	33.0%	13.5%	0.432	19.2%	13.8GB
CodeLlama-7b-Instruct	30.4%	19.0%	0.428	20.3%	13.0GB
Mistral-7B-Instruct-v0.3	27.7%	16.3%	0.443	18.5%	13.1GB



3.1 消融实验：量化方案对比（固定 Qwen2.5-Coder-7B-Instruct）

寻找模型性能与硬件算力约束之间的最优平衡点

➤ 控制变量：



相同的基座模型



相同数据



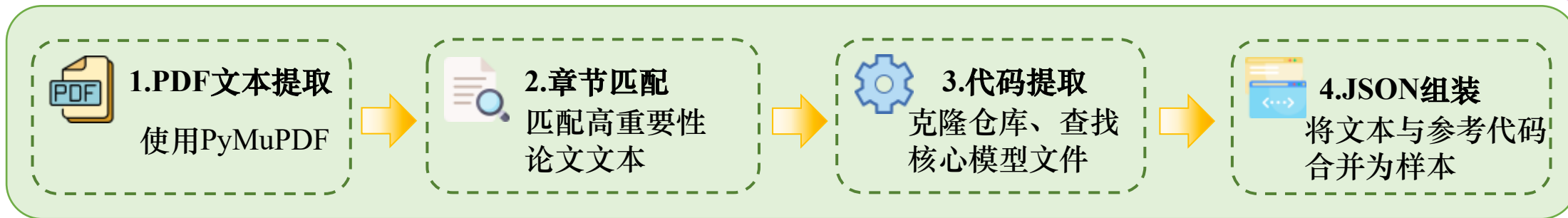
相同LoRA rank (r=8、alpha=16)

➤ 实验结果：

量化方案	模型性能评估				算力资源开销	
	XPR	SYR	CodeBERTScore	Overall	峰值显存	训练耗时
BF16 全参数微调	15.2%	35.1%	0.450	21.1%	31.5GB	45.3min
BF16 + LoRA	14.9%	32.9%	0.431	20.4%	18.2GB	14.9min
8-bit QLoRA	14.8%	32.0%	0.485	21.0%	14.2GB	4.0min
4-bit QLoRA	14.7%	34.7%	0.444	20.5%	13.2GB	2.6min

3.2 评测指标选取与评测流程

➤ 构建流程（75篇2025年论文）：



➤ 指标体系设计：

可运行维度



XPR（可执行率）：验证代码在沙盒中是否可运行



SAR（独立运行性）：与 XPR 类似，但在执行前移除 `PYTHONPATH` 环境变量，试图模拟更严格的隔离环境，但在实操中出现隔离不彻底的问题，故移除



SYR（语法正确率）：使用 Python 的 `ast.parse()` 函数对代码进行语法解析

论文忠实度



PCR（论文组件覆盖率）：GPT-4o-mini extract/match
 $PCR = \frac{\text{满足的检查项数}}{\text{总检查项数}}$



AAR（API 调用对齐率）：提取 PyTorch API 调用，计算两个 API 集合的 Jaccard 相似度



CodeBERTScore：计算生成代码与参考代码的语义相似度



3.3 微调结果

训练配置

- 📌 基础模型: Qwen2.5-Coder-7B-Instruct (~15GB)
- 📌 微调方式: 4-bit QLoRA (量化 + LoRA 适配器)
- 📌 关键配置:
 - LoRA target: all (所有线性层)
 - Epochs: 8
 - Learning rate: $2e-4$
 - Batch size: $2 \times$ 梯度累积 8
 - 截断长度: 2048 tokens
 - 混合精度: FP16

训练结果

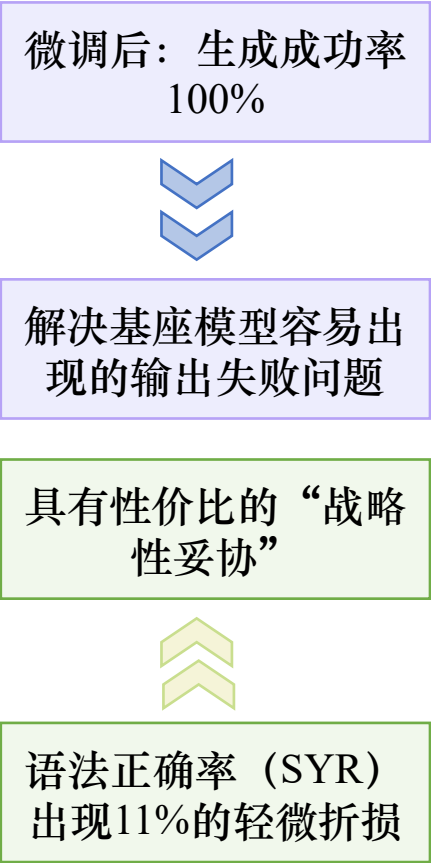
- 📌 训练损失收敛至 0.5489
- 📌 训练耗时: 0:09:41, 约 10 分钟
(90 条数据, 8 epochs)
- 📌 训练显存: 约11GB

LoRA合并

- 适配器 → 完整模型 (~15GB)
- 导出格式: safetensors



3.4 基准测试与对比分析



Base vs 微调后对照结果：

指标	Baseline	Fine-tuned	提升
XPR	9.38%	14.67%	+56%
SAR	9.38%	14.67%	+56%
SYR	39.06%	34.67%	-11%
PCR	9.77%	16.19%	+66%
AAR	3.16%	5.08%	+61%
CodeBERTScore	0.380	0.444	+17%
总分	16.11%	20.54%	+27%
生成成功率	64/75	75/75	+100%



4

增量工作和交付结果



4.1 核心增量1：代码蒸馏机制

step 1:

调用api由一个大模型（GPT-4o）对整个仓库做抽取与蒸馏（伪代码）



step 2:

分解主要工具链（代码函数与封装），参考生成的核心摘要，分别生成对应的代码补全决策指导，交由两个学生模型分别做生成和修正

生成模型

老师模型分步输入该抽象级函数块的伪代码和实现逻辑，基于此补全代码，并给出测试脚本



修正模型

基于生成模型补全后的代码，在沙盒中运行和验证，并根据调试结果进行修正（已跑通，具体细节待调试）

4.1 核心增量2：图谱诊断与沙盒动态验证引擎

xKG的构建与抽取

语义融合：代码结构→论文意图

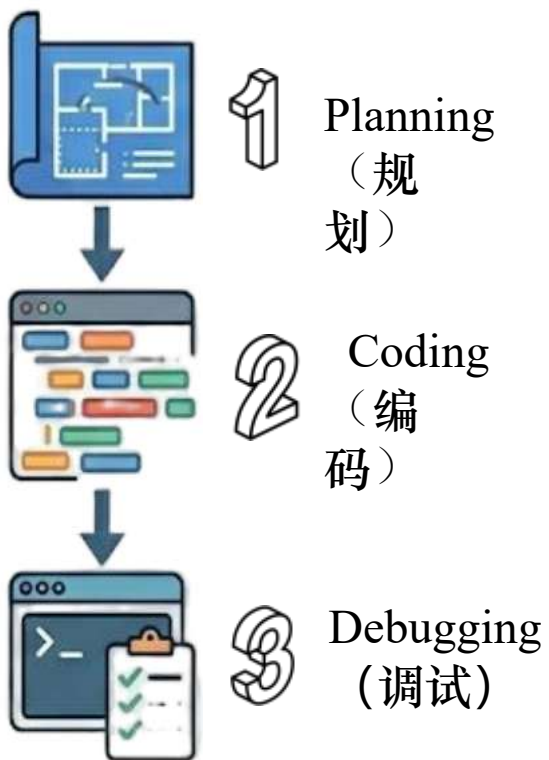
```
```json
{
 "paper_title": "AttentionCalibration",
 "abstract": "...",
 "techniques": [
 {
 "name": "Core Model",
 "type": "Methodology",
 "description": "Core algorithm implementation from core_model.py...",
 "components": [],
 "code": {
 "implementation": "import torch\n...",
 "documentation": "",
 "package": ["torch"]
 }
 }
]
}
```

### 沙盒环境下的xKG自调试

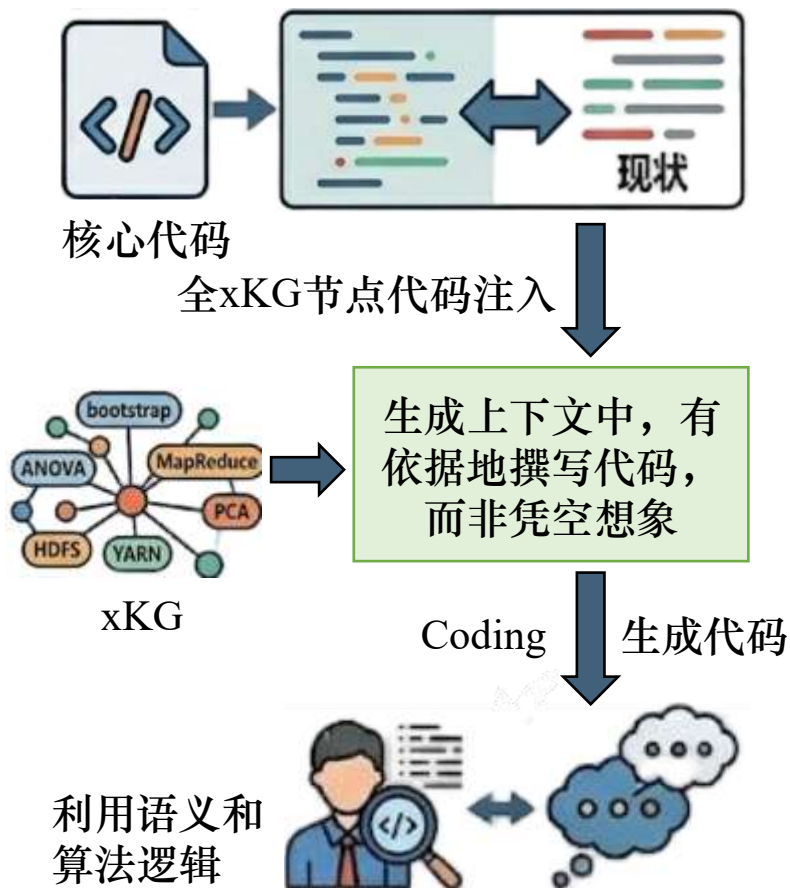


## 4.1 核心增量3：仓库级代码生成与一致性修复

### 沿用Paper2Code三步法



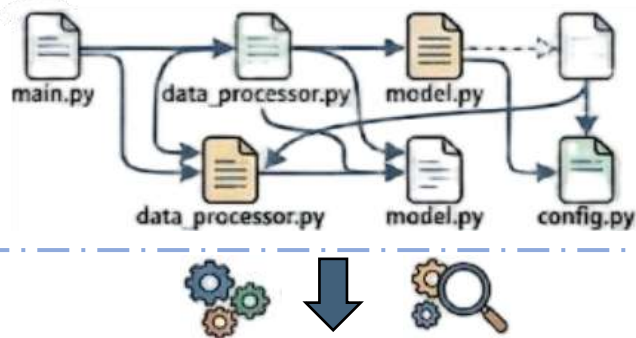
### 第二步Coding中的xKG注入



### 跨文件符号依赖方法与Repo-level修复

aider项目：Repo-level一致性修复

设计：分析跨文件符号依赖



Repo-level修复机制

修复：

- import报错
- 接口不一致

生成稳定、一致的代码



- [illegible]



## 4.2 交付结果演示

- 这是一个真实的xKG结果
- 其中包含两个技术节点

```
{
 "paper_title": "MyPaper",
 "abstract": "A challenging problem in many modern machine learning tasks is to process weight-space features, i.e., to transform or ext",
 "techniques": [
 {
 "name": "Core Model",
 "type": "Methodology",
 "description": "Core algorithm implementation from core_model.py, generated by Qwen2.5-Coder-7B SFT for paper 'MyPaper'.",
 "components": [],
 "code": {
 "implementation": "import torch\nimport torch.nn as nn\nfrom typing import List, Tuple\n\nclass EquivariantLayer(nn.Module):\n",
 "documentation": "",
 "package": [
 "torch"
]
 },
 "verified": true
 },
 {
 "name": "Core loss",
 "type": "Technique",
 "description": "Core algorithm implementation from core_loss.py, generated by Qwen2.5-Coder-7B SFT for paper 'MyPaper'.",
 "components": [],
 "code": {
 "implementation": "import torch\nimport torch.nn as nn\n\nclass CustomLoss(nn.Module):\n def __init__(self):\n super(Cu",
 "documentation": "",
 "package": [
 "torch"
]
 },
 "verified": true
 }
],
 "validation_stats": {
 "total": 2,
 "passed": 2,
 "pruned": 0,
 "debugged": 0
 }
}
```



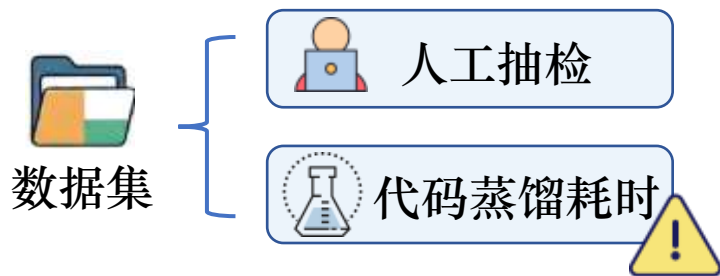
# 5

## 未来展望与局限分析

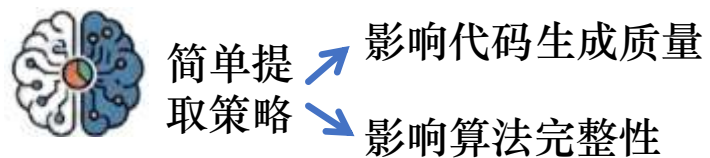
## 5.1 局限与分析

### 针对数据集

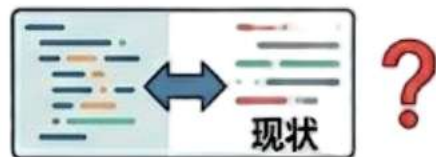
Q1: 数据集过小



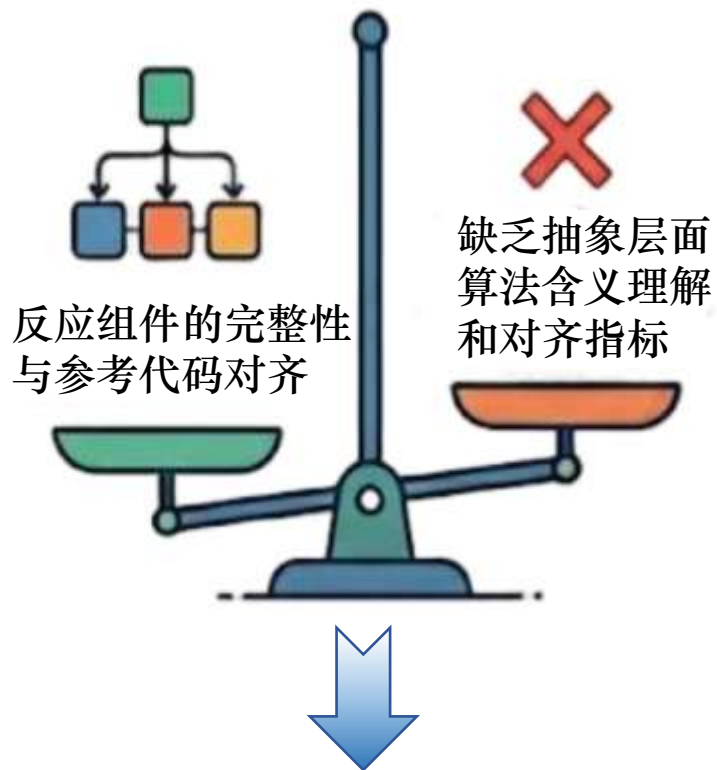
Q2: 数据集的核心内容是否能反映完整框架



代码和核心内容对齐与有效性验证

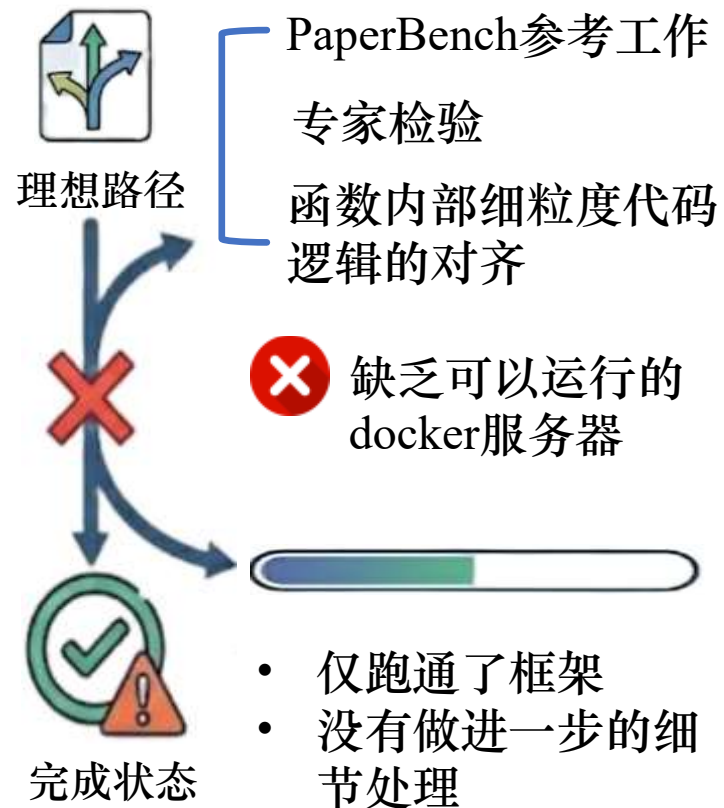


### 针对指标与评估流程



现状: 组件对齐与代码对齐验证

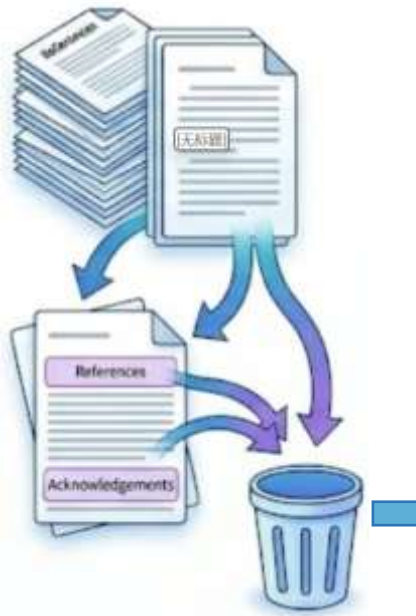
### 完成情况受限





## 5.2 未来展望：论文核心文本的提取

### 1. 全局去噪



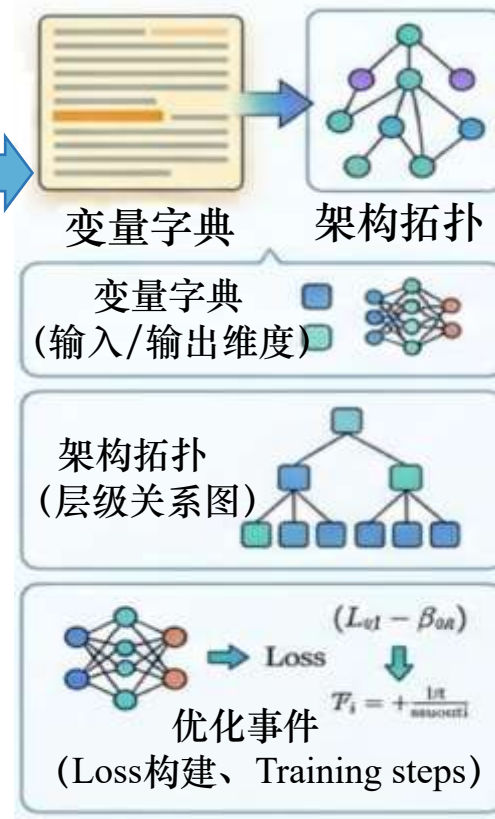
去除参考文献、致谢等明确无用的部分

### 2. 切块与打分



保留含有高度“算法语义”的块

### 3. 拓扑提取与逻辑重构



### 4. 思维链精馏提取



生成一份详尽的、消除歧义的算法伪代码 (Pseudo-code) 或 IR (Intermediate Representation)

参考文献：《From text to idea embeddings: a review on quintessential information extraction》、  
《A Comprehensive Survey on Automatic Text Summarization with Exploration of LLM-Based Methods》



# 感谢

Thanks

- 建模：蒋佳媛、但杰
- 数据集：蒋佳媛（有效性验证）、高鑫彤
- 微调：但杰（设计）、高鑫彤、蒋佳媛、徐媛（做实验）
- 评测：徐媛、高鑫彤（设计）、但杰（实验）
- 后续：但杰
- ppt制作：高鑫彤

2026.6.3